

PRACTICAL-05

AIM: To Implementation of a knapsack problem using dynamic programming.

Knapsack

ALGORITHM:

Input :

- * Number of items n
- * weight $w[i]$ and $v[i]$ of each item
- * Knapsack capacity w .

Output :

- * maximum value that can be obtained
- * Selected items.

Step 1: Start

Step 2: Read the number of item n and Knapsack capacity w .

Step 3: Read the weight $w[i]$ and value $v[i]$ of each item

Step 4: initialize the first row and first column of the DP table to 0.

Step 5: For each item $i=1$ to n , repeat step 6.

Step 6: For each capacity $j=1$ to w :

if $w[i] \leq j$, calculate:

$$dp[i][j] = \max(v[i] + dp[i-1][j-w], dp[i-1][j])$$

otherwise

$$dp[i][j] = dp[i-1][j]$$

Step 7: The maximum value is stored in $dp[n][w]$

Step 8: Trace the dp table from $dp[n][w]$ backward to find the selected items

Step 9: Display the DP table, max value and selected item

Step 10: Stop

PROGRAM:

```
#include <iostream>

using namespace std;

class Knapsack {
private:
    int w[10];
    int v[10];
    int n;
    int W;
    int dp[10][100];

public:
    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        for (int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    void solve() {
        for (int i = 0; i <= n; i++) dp[i][0] = 0;
        for (int j = 0; j <= W; j++) dp[0][j] = 0;
```

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= W; j++) {  
        if (w[i] <= j) {  
            dp[i][j] = max(v[i] + dp[i-1][j-w[i]], dp[i-1][j]);  
        } else {  
            dp[i][j] = dp[i-1][j];  
        }  
    }  
}
```

```
void printTable() {  
    cout << "DP Table:" << endl;  
    cout << "  ";  
    for (int j = 0; j <= W; j++) cout << j << " ";  
    cout << endl;  
    for (int i = 0; i <= n; i++) {  
        cout << "i=" << i << " ";  
        for (int j = 0; j <= W; j++) cout << dp[i][j] << " ";  
        cout << endl;  
    }  
}
```

```
void findItems() {  
    cout << "\nSelected Items:" << endl;  
    int j = W;  
    for (int i = n; i >= 1; i--) {  
        if (dp[i][j] != dp[i-1][j]) {  
            cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ") TAKEN" << endl;  
            j = j - w[i];  
        }  
    }
```

```
        j = j - w[i];
    } else {
        cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ") NOT taken" <<
endl;
    }

}

}

void showResult() {
    cout << "\nMaximum Value = " << dp[n][W] << endl;
}

};

int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

OUTPUT:

```
Enter number of items : 6
Enter knapsack capacity : 10
Item 1 weight and value : 5
3
Item 2 weight and value : 2
8
Item 3 weight and value : 3
7
Item 4 weight and value : 1
5
Item 5 weight and value : 4
6
Item 6 weight and value : 7
1
DP Table:
  0 1 2 3 4 5 6 7 8 9 10
i=0 0 0 0 0 0 0 0 0 0 0
i=1 0 0 0 0 0 3 3 3 3 3
i=2 0 0 8 8 8 8 8 11 11 11
i=3 0 0 8 8 8 15 15 15 15 18
i=4 0 5 8 13 13 15 20 20 20 20
i=5 0 5 8 13 13 15 20 20 20 21
i=6 0 5 8 13 13 15 20 20 20 21
Maximum Value = 26

Selected Items:
Item 6 (w=7, v=1) NOT taken
Item 5 (w=4, v=6) TAKEN
Item 4 (w=1, v=5) TAKEN
Item 3 (w=3, v=7) TAKEN
Item 2 (w=2, v=8) TAKEN
Item 1 (w=5, v=3) NOT taken
```

TIME COMPLEXITY:

```
for (int i=1; i<=n; i++) → n
{
    for (int j=1; j<=w; j++) → w
    {
        // ...
    }
}
T(n, w) = n x w
```

Best case :

Even if the items are not suitable
program still fills the complete Dp table
 $O(nw)$

Average case :

The program still fills all new entries
 $O(nw)$

worst case :

The program still fills all $n \times w$ entries
 $O(nw)$

CONCLUSION: The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the maximum value that can be obtained within the given knapsack capacity by considering whether each item should be selected or not selected. The time complexity of the algorithm is $O(nW)$ and the space complexity is $O(nW)$.